

Materia: Programación Imperativa	Código: 72.31
Créditos: 9	Carga horaria total: 153
Departamento: Sistemas Digitales y Datos	Año: 2023
Carrera de: Ingeniería en Informática / Ingeniería Mecánica /	
Fecha última actualización: 2/5/2023 14:44:31	

➤ Carga horaria:

153	Horas totales
85	hs. teóricas
34	hs. prácticas
34	hs. de laboratorio

9	Horas semanales
9	hs. presencial
0	hs. a distancia

➤ Contenidos mínimos:

Características del Paradigma Imperativo. El algoritmo como modelo de proceso computacional. Prueba y depuración del software. Sintaxis y semántica básica de lenguajes de alto nivel. Tipos de datos simples, operadores y expresiones. Variables y asignación. Entrada y salida de datos. Estructuras de control. Concepto de función. Uso y construcción de bibliotecas. Recursividad. Concepto de punteros y referencias. Tipos de datos estructurados: arreglos, cadena de caracteres y registros. Manejo de archivos. Taller de aplicación con un lenguaje imperativo tipo lenguaje ANSI C bajo Linux.

➤ Presentación de la materia:

Esta materia se dicta en el segundo cuatrimestre de primer año, y es la primer materia de programación de la carrera. Los alumnos deben contar con los conocimientos básicos de Álgebra y de sistemas de numeración.

Si bien usaremos del lenguaje C, no debe verse como un curso de programación en C sino como un curso de programación, de forma tal que al finalizar el mismo los alumnos no solo serán capaces de escribir bibliotecas y programas en C correctos y con buen estilo sino también poder adaptarlos a otros lenguajes similares.

➤ Objetivos de aprendizaje:

Los objetivos a lograr son 3:

- ☐ Aplicar las técnicas de diseño modularizado para descomponer problemas.
- ☐ Explicar el funcionamiento de algoritmos y analizar su correctitud diseñando pruebas efectivas para su evaluación.
- ☐ Escribir algoritmos correctos, eficientes y con buen estilo de programación para entornos multiplataforma.

➤ Contenidos:

Título	Descripción
Introducción a la Programación	Programación modularizada y estructurada. Buen estilo de programación. Concepto de Algoritmo. Pseudocódigo y refinamiento. Lenguajes de alto nivel. Compiladores y preprocesadores. Testeo de Software: caja negra, caja blanca y caja de Pandora.
Tipos, Operadores y Expresiones	Tipos y tamaño de datos. Constantes. Declaraciones. Operadores: aritméticos, relacionales, lógicos, de incremento y decremento, para manejo de bits y de conversión. Operadores de asignación y expresiones. Precedencia de operadores. Instrucciones básicas de E/S: printf y getchar. Redireccionamiento de la entrada y de la salida.
Estructuras de Control	Preposiciones y bloques. Almacenamiento y alcance de variables. Variables externas y estáticas. Estructuras de decisión: if-else y switch. Estructuras de repetición: while, for y do-while. Uso de break y continue.
Funciones.	Concepto de Función. Prototipación. Parámetros de entrada. Biblioteca estándar de C (funciones básicas). Archivos header. El Preprocesador de C. Parámetros de entrada y salida. Entrada formateada: scanf y sscanf. Construcción de Bibliotecas. Funciones con argumentos variables. Recursividad
Arreglos	Arreglos unidimensionales. Arreglos de caracteres. Ordenamiento de arreglos. Búsqueda en arreglos unidimensionales. Arreglos multidimensionales
Archivos	Concepto de flujo de datos: stream. Acceso a archivo. Archivos binarios y de texto. Manejo de errores. Procesamiento de archivos en Lenguaje C
Conceptos avanzados del lenguaje C	Punteros. Aritmética de punteros. Punteros a caracteres. Estructuras. Uniones. Uso del Heap. Punteros a función. Punteros genéricos. Implementación de TADs (Tipo Abstracto de Datos): pilas, colas, listas.

➤ Estrategias de enseñanza:

En las clases teóricas, que se dictan en modalidad virtual sincrónica, se presentan los temas y se realizan ejercicios cortos para verificar que los alumnos hayan comprendido y asimilado los conceptos. Al ser online se aprovecha también el uso de "polls" que provee Campus para realizar preguntas conceptuales, no sólo a fin de reemplazar el "¿se entendió?" sino también para captar la atención del alumno.

En las clases prácticas, presenciales, se presentan problemas que permitan que los alumnos apliquen los temas vistos en las teóricas, y se discuten junto con los alumnos las distintas formas de resolverlos, asegurando que cumplan con los 4 requisitos planteados por la Cátedra: cada código debe ser correcto, eficiente, claro y general.

➤ Actividades:

Título	Descripción
Clases teóricas	En las clases teóricas se desarrollan los temas que integran el programa de la materia incluyendo pequeños ejemplos de aplicación. Se espera que el alumno haya leído el tema del día previamente para que la clase se vea enriquecida con su participación. Cabe aclarar que las clases no reemplazan a la bibliografía sino que la complementan y actualizan.
Clases prácticas	El objetivo de estas clases es profundizar, desde el punto de vista práctico, los conceptos desarrollados en las clases teóricas y también desarrollar ciertos temas no vistos en clase teórica por ser de corte netamente práctico. Durante la clase se plantean ejercicios para que los alumnos resuelvan en forma individual y/o grupal (según cada caso) en un tiempo acotado, y posteriormente se realiza una puesta en común de resultados para evaluar ventajas y desventajas de todas las propuestas de solución planteadas. Además se dan lineamientos que facilitan el desarrollo de las guías de trabajos prácticos. Se considera fundamental que el alumno participe activamente en las discusiones propuestas para cada tema a fin de poder adquirir un buen criterio en el diseño e implementación de algoritmos, con claridad y eficiencia.
Laboratorios	En las clases de laboratorio los alumnos aplican los conocimientos adquiridos a través de la resolución de los ejercicios presentes en las guías de trabajos prácticos. Dichos ejercicios están especialmente diseñados para ayudar al aprendizaje de los puntos importantes en la materia. La modalidad de trabajo es individual y se espera que el alumno aplique los siguientes pasos en la resolución de ejercicios: - Análisis del problema - Planteo de solución en papel - Escritura de un programa que se ajuste a la solución en papel. - Verificación del programa en papel (aplicando métodos de prueba) - Escritura, compilación y prueba del programa en máquina. Los docentes estarán a disposición de los alumnos para resolver las dudas que surgen en este proceso y al finalizar la clase podrán solicitar la entrega de ejercicios indicados al comienzo de la misma, a modo de seguimiento. La entrega de dichos ejercicios es optativa y se calificará con

	una nota de concepto.
Trabajo Práctico Especial	Para el examen final, los alumnos deberán desarrollar y presentar un trabajo práctico especial que reflejará los conceptos adquiridos durante el cursado de la materia. Dicho trabajo será realizado en grupos. El trabajo especial consiste en la implementación de una biblioteca en lenguaje C. La funcionalidad requerida en el trabajo de programación será asignada a cada grupo por la cátedra. En la fecha del examen final, el trabajo práctico especial será presentado a través de un coloquio individual y/o grupal.

➤ Recursos didácticos:

Para las clases online disponen de las presentaciones previamente publicadas. En las clases de taller se utilizan presentaciones, proyector y pizarrón.

➤ Modalidad de evaluación y aprobación:

Modalidad de evaluación:

La evaluación de la materia se realiza a través de:

- Autoevaluaciones (publicadas en Campus). Todos los alumnos deben completar las autoevaluaciones antes de su fecha de vencimiento.
- Dos exámenes parciales obligatorios e individuales, con preguntas teóricas y ejercicios prácticos. Estos parciales se aprueban con un mínimo de 5 (cinco) puntos.

En el caso de que el alumno desaprobe un parcial, contara con una instancia de recuperatorio. Ambos parciales son recuperables. En el caso de aprobar el recuperatorio de un parcial la nota del mismo se obtiene como: $0.25 * \text{Nota del parcial desaprobado} + 0.75 * \text{Nota del recuperatorio aprobado}$.

La nota de cursada es igual a $0.9 * \text{Nota de Parciales} + 0.1 * \text{Notas de autoevaluaciones}$

Requisitos de aprobación:

Para aprobar la cursada los alumnos deben:

- Aprobar ambos parciales (o sus recuperatorios correspondientes).
- Completar las autoevaluaciones online.

Para aprobar la materia, los alumnos que ya hayan aprobado la cursada deben rendir el examen final de la materia que, como se explicó más arriba, consiste en la presentación escrita y oral del Trabajo Final Especial de la materia.

➤ Bibliografía obligatoria:

N°	Descripción
----	-------------

1	Kernighan, B. y Ritchie, D., "El Lenguaje de Programación C", Prentice Hall
---	---

➤ **Bibliografía complementaria:**

N°	Descripción
1	B. Kernighan & R.Pike , "The Practice of Programming", Addison Wesley
2	Roberts, E., "The Art and Science of C", Addison Wesley
3	Deitel, H. & Deitel, P., "Cómo Programar en C/C++", Prentice Hall
4	Samuel P. Harbison y Guy Steel Jr., "C: a reference manual", Prentice Hall